Laporan Praktikum Pemrograman Terapan - 06

Nama: Jonathan Valent Waluya NRP: 2472010

Dokumentasi Pengerjaan & Pengumpulan

1. Source Code Aplikasi

1. Aplikasi web sederhana: `index.html`
2. Aplikasi Laravel: ada di folder `laravel-app/`

2. Dockerfile yang Digunakan

A. Dockerfile Aplikasi Web Sederhana (`./Dockerfile`)

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80
```

B. Dockerfile Aplikasi Laravel (`./laravel-app/Dockerfile`)

```
FROM php:8.4-cli

RUN apt-get update && apt-get install -y \
    libpng-dev \
    libonig-dev \
    libxml2-dev \
    zip \
    unzip \
    curl \
    sqlite3 \
    libsqlite3-dev

RUN docker-php-ext-install pdo pdo_sqlite mbstring

WORKDIR /app

COPY . /app
```

```
EXPOSE 8000

CMD ["php", "artisan", "serve", "--host=0.0.0.0", "--port=8000"]
```

---

3. Screenshot Aplikasi di Browser

1. **Aplikasi Web Sederhana**
   - URL: `http://localhost:8080`
   - Halaman HTML statis yang menampilkan nama, NRP, dan keterangan bahwa web server yang dipakai adalah Nginx. *(Sertakan screenshot browser saat membuka URL di atas)*

2. **Aplikasi Laravel**
   - URL: `http://localhost:8000`
   - Halaman landing page Laravel yang berjalan lewat container PHP CLI. *(Sertakan screenshot browser saat membuka URL di atas)*

---

4. Screenshot Hasil `docker images` dan `docker ps`

Output `docker ps`:

```
CONTAINER ID   IMAGE                 COMMAND                  CREATED             STATUS
be9dc865ea9c   p06-laravel-2472010   "docker-php-entrypoi…"   About an hour ago   Up About an hou
8eb74a3f1f9b   p06-web-2472010       "/docker-entrypoint.…"   2 hours ago         Up 2 hours
```

Output `docker images`:

```
IMAGE                          ID             DISK USAGE   CONTENT SIZE
p06-laravel-2472010:latest     ff46b182f1fa   894MB        219MB
p06-web-2472010:latest         037d1fc8bf49   93.6MB       26.3MB
nginx:alpine                   df221db836e1   94.4MB       27.2MB
```

(Sertakan screenshot terminal setelah menjalankan perintah di atas)

---

5. Proses Pengerjaan

Langkah 1: Aplikasi Web Sederhana

1. Buat file `index.html` yang isinya HTML sederhana.
2. Buat `Dockerfile` dengan base image `nginx:alpine`, lalu copy `index.html` ke folder web root Nginx yaitu `/usr/share/nginx/html/index.html`.
3. Build image dengan perintah: `docker build -t p06-web-2472010 .`
4. Jalankan container dengan port forwarding: `docker run -d -p 8080:80 --name p06-web-container p06-web-2472010`
5. Coba buka `http://localhost:8080` di browser.

Langkah 2: Aplikasi Laravel

1. Buat project Laravel baru di folder `laravel-app` pakai Composer: `composer create-project --prefer-dist laravel/laravel:^10.0 laravel-app --no-install`
2. Install dependensi dengan `composer install`, lalu buat app key dengan `php artisan key:generate`.
3. Buat `Dockerfile` di dalam folder `laravel-app/`. Isinya PHP, ekstensi PDO SQLite, dan expose port 8000.
4. Build image Laravel: `docker build -t p06-laravel-2472010 .`
5. Jalankan container: `docker run -d -p 8000:8000 --name p06-laravel-container p06-laravel-2472010`
6. Coba buka `http://localhost:8000` di browser.

---

6. Hasil Eksplorasi Laravel

Yang Berhasil:

1. Membuat project Laravel dan menginstall dependensi vendor.
2. Membuat Dockerfile berbasis PHP CLI dengan ekstensi yang dibutuhkan (`pdo`, `pdo_sqlite`, `mbstring`).
3. Membuat Docker image Laravel (`p06-laravel-2472010`).
4. Menjalankan container Laravel di background dan membuka landing page Laravel dari browser.

Kendala dan Pesan Error:

1. **Versi PHP tidak cocok (Platform Check)**
    - Pesan error:

```
Fatal error: Uncaught RuntimeException: Composer detected issues in your platform:
Your Composer dependencies require a PHP version ">= 8.4.1". You are running 8.2.35.
in /app/vendor/composer/platform_check.php:22
```

- Penyebab: Awalnya saya pakai image `php:8.2-cli`, padahal dependensi Composer yang terinstall butuh PHP minimal 8.4.1.
- Solusi: Ganti base image di `Dockerfile` jadi `php:8.4-cli`, lalu build ulang image.

2. **Koneksi ditolak saat diakses dari host**
   - Pesan error:

     ```
     Unable to connect to the remote server / Connection refused
     ```

   - Penyebab: `php artisan serve` secara default hanya listen di `127.0.0.1` (di dalam container), jadi akses dari luar container lewat port forwarding ditolak.
   - Solusi: Tambahkan `--host=0.0.0.0` di perintah CMD pada `Dockerfile`: `CMD ["php", "artisan", "serve", "--host=0.0.0.0", "--port=8000"]`

3. **Composer memblokir package karena security advisories**
   - Pesan error:

     ```
     Your requirements could not be resolved to an installable set of packages.
     Problem 1: Root composer.json requires laravel/framework ^10.0 ... affected by securi
     ```

   - Penyebab: Composer versi terbaru otomatis mengaktifkan audit keamanan, jadi package yang punya CVE aktif tidak bisa diinstall.
   - Solusi: Jalankan `composer config audit.block-insecure false` sebelum `composer install`.

Pertanyaan Refleksi

1. **Apa perbedaan menjalankan aplikasi langsung di komputer dengan menjalankannya di Docker?**

   Kalau dijalankan langsung di komputer, kita harus install sendiri semua kebutuhannya (web server, PHP, ekstensi, dll) di sistem operasi. Cara ini gampang bikin konflik versi antar project, atau bikin aplikasi jalan di komputer satu orang tapi error di komputer lain ("it works on my machine"). Kalau pakai Docker, aplikasi dan dependensinya dibungkus dalam satu container yang terpisah, jadi hasilnya sama di komputer mana pun dan tidak mengubah pengaturan OS kita.

2. **Mengapa aplikasi Laravel membutuhkan lingkungan yang berbeda dari aplikasi HTML statis?**

HTML statis cuma berisi file teks, CSS, dan file statis lain, jadi cukup disajikan oleh web server seperti Nginx ke browser. Laravel itu framework PHP yang dinamis. Jadi butuh interpreter PHP untuk menjalankan kode di sisi server, akses baca-tulis ke folder `storage` dan `bootstrap/cache`, koneksi database, Composer untuk autoload dependensi, dan sistem routing.

3. **Apa fungsi PHP dan Composer dalam Laravel?**

    - **PHP:** bahasa pemrograman sekaligus runtime yang menjalankan semua logika aplikasi Laravel, seperti controller, model, routing, dan template Blade.
    - **Composer:** package manager untuk PHP. Fungsinya mengunduh dan mengatur versi library yang dibutuhkan Laravel, dan juga menyediakan autoloading (PSR-4) supaya class di aplikasi bisa dipanggil otomatis.

4. **Apa perbedaan image dan container dalam proses menjalankan aplikasi Laravel?**

    - **Image:** semacam cetak biru yang sifatnya read-only. Isinya OS dasar, PHP, ekstensi, dependensi vendor, dan kode Laravel. Image sendiri tidak berjalan, hanya dipakai untuk membuat container.
    - **Container:** hasil "menjalankan" sebuah image. Container punya lapisan read-write di atas image dan punya proses, memori, serta IP virtual sendiri. Di sinilah `php artisan serve` benar-benar berjalan.

5. **Mengapa konfigurasi database perlu diperhatikan ketika Laravel dijalankan dalam Docker?**

    Karena jaringan di dalam container terpisah dari host, jadi `localhost` atau `127.0.0.1` merujuk ke container itu sendiri, bukan ke komputer kita atau container lain. Kalau pakai SQLite yang berupa file, kita harus pastikan path file dan izin tulis foldernya benar di dalam container. Kalau database-nya ada di luar container (misalnya MySQL di host), host di file `.env` harus diganti, misalnya jadi `host.docker.internal` atau nama service di network Docker, supaya container bisa terhubung.